

NAME: K. Yamini

REG.NO:192572181

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

***CO5:** Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.(BTL 5)*

Assessment Tool Weightage: 30%

QUESTIONS: 5

Duration: 1 Hour

Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I K.Yamini(192572181) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student:



Annexure A

Reverse Engineering

1. System Decomposition & Analysis

Analyze an existing AI-based system and reverse engineer its architecture, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, reconstruct its system architecture and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to identify the algorithms, data processing steps, and decision-making logic used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and map its data flow, processing pipeline, and communication between components.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and propose enhancements or redesign strategies to improve scalability, efficiency, and performance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Understanding of Industry Problem	20% (2)	/2	/2	/2	/2	/2	/10
Application of Engineering Concepts	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Solution Design	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Use of Tools	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Analysis	10% (1)	/1	/1	/1	/1	/1	/5
Professionalism	5% (0.5)	/0.5	/0.5	/0.5	/0.5	/0.5	/2.5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/50

1. Analyze an existing AI-based system and reverse engineer its architecture, identifying key components, data flow, and underlying algorithms.

Example: AI-Based Image Classification System

An AI-based image classification system is designed to identify and classify an input image into a particular category. Reverse engineering helps us understand the internal structure,

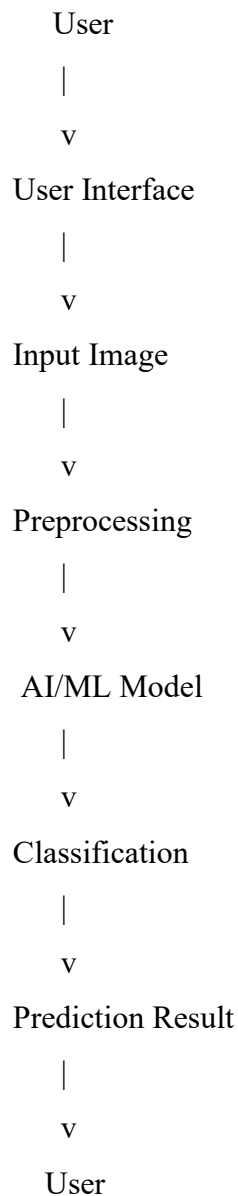
components, data flow, and algorithms of an existing system without depending only on its original design documents.

1. System Components

The major components are:

1. **User Interface** – Accepts an image from the user.
2. **Input Processing Module** – Reads and validates the image.
3. **Preprocessing Module** – Resizes, normalizes, and prepares the image.
4. **Feature Extraction / AI Model** – Extracts important features from the image.
5. **Classification Module** – Predicts the class of the image.
6. **Database** – Stores images, predictions, and related information.
7. **Output Module** – Displays the predicted result.

2. Architecture



3. Data Flow

The image is first provided by the user. The preprocessing module resizes and normalizes the

image. The processed image is given to the AI model. The model extracts useful features and performs classification. Finally, the predicted class is returned to the user.

4. Underlying Algorithms

A deep-learning-based system may use:

- **Convolutional Neural Network (CNN)** for image feature extraction.
- **ReLU** for activation.
- **Pooling** for reducing feature dimensions.
- **Softmax** for calculating class probabilities.
- **Backpropagation** during model training.

5. Reverse Engineering Process

The system can be analyzed by:

- Observing input and output.
- Identifying individual modules.
- Studying data movement between modules.
- Examining model behavior.
- Analyzing logs and API calls.
- Testing different inputs and comparing outputs.

6. Improvements

The system can be improved by:

- Using a larger and better-quality dataset.
- Optimizing the AI model.
- Reducing unnecessary preprocessing.
- Using caching for repeated inputs.
- Improving database performance.

Conclusion

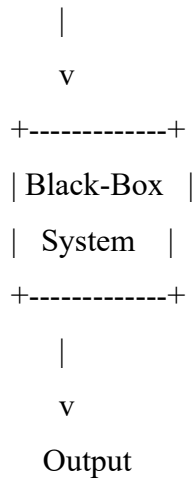
Reverse engineering allows us to understand the architecture, components, data flow, and algorithms of an existing AI system. It also helps identify limitations and provides opportunities for improving system performance and efficiency.

2. Given a black-box software system, reconstruct its system architecture and explain the interaction between modules using appropriate diagrams.

A **black-box software system** is a system whose internal implementation is unknown. We can understand its behavior by observing its inputs, outputs, and interactions with external components. Reverse engineering helps reconstruct its architecture.

1. Black-Box View

Input



Initially, we do not know what happens inside the system.

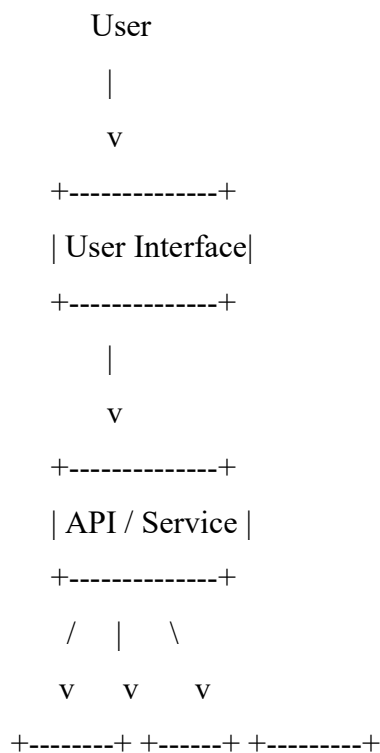
2. Information Collection

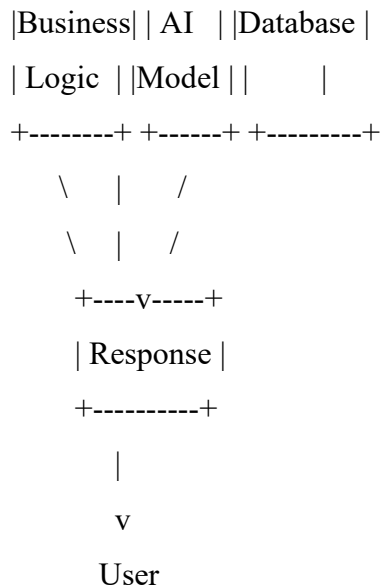
We examine:

- Input formats
- Output formats
- System responses
- APIs
- Logs
- Database interactions
- Error messages
- Communication between components

3. Reconstructed Architecture

After analysis, the system may be represented as:





4. Module Interaction

User Interface → API:

The user sends a request through the interface.

API → Business Logic:

The API validates and processes the request.

Business Logic → AI Model:

The required data is passed to the AI model.

AI Model → Database:

The model may retrieve training/configuration data or store prediction information.

Database → Business Logic:

Required information is returned.

Business Logic → API:

The processed result is returned.

API → User Interface:

The final output is displayed to the user.

5. Reverse Engineering Techniques

The following techniques can be used:

- Dynamic analysis
- Static analysis, where source/binary information is available
- API monitoring
- Log analysis
- Input-output testing
- Dependency analysis
- Database analysis

6. Benefits

Architecture reconstruction helps to:

- Understand an unknown system.
- Maintain legacy software.
- Identify dependencies.
- Detect performance problems.
- Plan modernization.
- Improve scalability and security.

Conclusion

By systematically analyzing inputs, outputs, communication, logs, and dependencies, the architecture of a black-box system can be reconstructed. Appropriate diagrams make the relationships between modules clear and understandable.

3. Reverse engineer a machine learning application to identify the algorithms, data processing steps, and decision-making logic used in the system.

Reverse engineering a machine learning application means analyzing its behavior and components to determine the **data processing steps, machine-learning algorithms, and decision-making logic** used by the application.

1. Machine Learning Application

Consider a **student performance prediction system** that predicts whether a student is likely to pass or fail.

2. System Flow

Student Data

|

v

Data Collection

|

v

Data Cleaning

|

v

Feature Selection

|

v

Data Preprocessing

|

v
ML Model

|
v
Prediction

|
v
Decision

|
v
Pass / Fail

3. Data Processing Steps

Step 1 – Data Collection

The system collects information such as:

- Attendance
- Internal marks
- Assignment marks
- Previous performance

Step 2 – Data Cleaning

Missing or incorrect values are identified and handled.

Step 3 – Feature Selection

Important features are selected for prediction.

Step 4 – Data Transformation

Numerical values may be normalized or standardized.

Step 5 – Model Prediction

The processed data is given to the trained ML model.

4. Possible Algorithm

A classification algorithm such as **Decision Tree** can be used.

Example:

```
Attendance > 75%?
  /      \
Yes       No
 /      \
Internal Marks >50?  Fail
 /      \
Yes      No
```

/ \
Pass Fail

5. Decision-Making Logic

The model examines the input features and follows learned rules to produce a prediction.

For example:

- High attendance + high internal marks → **Pass**
- Low attendance + low internal marks → **Fail**

6. Reverse Engineering Methods

The algorithm can be investigated using:

- Input-output testing
- Model behavior analysis
- Feature importance
- Logs
- Model metadata
- Performance evaluation
- Visualization of decision rules

7. Evaluation

The reconstructed model can be tested using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

8. Improvements

The model can be improved by:

- Collecting more representative data.
- Removing irrelevant features.
- Handling missing values properly.
- Selecting a suitable algorithm.
- Retraining the model periodically.

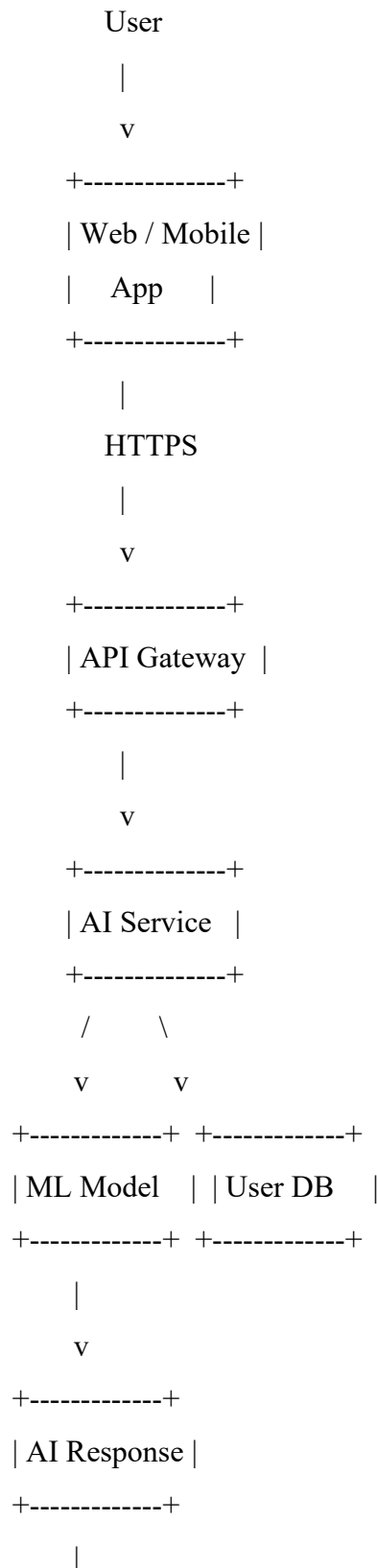
Conclusion

Reverse engineering helps identify how data moves through a machine-learning application, which algorithm is being used, and how the final decision is generated. This improves understanding, debugging, and future development.

4. Examine an existing distributed AI application and map its data flow, processing pipeline, and communication between components.

A distributed AI application consists of multiple components running on different systems or services. These components communicate with each other through networks or APIs. Reverse engineering helps map the complete data flow and processing pipeline.

Example: Distributed AI Chatbot



v

User

1. Data Flow

Step 1 – User Input

The user enters a question through a web or mobile application.

Step 2 – API Gateway

The request is sent to the API gateway through a network connection.

Step 3 – AI Service

The AI service receives and processes the request.

Step 4 – Model Processing

The AI model analyzes the input and generates a response.

Step 5 – Database

User information or conversation history may be retrieved from the database.

Step 6 – Response

The AI-generated response is sent back through the API gateway to the user.

2. Processing Pipeline

Input

↓

Validation

↓

Preprocessing

↓

Model Inference

↓

Post-processing

↓

Response

3. Communication Between Components

Components may communicate using:

- REST APIs
- HTTP/HTTPS
- Message queues
- Database connections
- Internal service APIs

4. Reverse Engineering

During analysis, we identify:

- Service boundaries
- API endpoints
- Data formats
- Communication protocols
- Database interactions
- Processing stages
- Dependencies

5. Problems That Can Be Identified

Reverse engineering can reveal:

- Network delays
- Bottlenecks
- Failure points
- Excessive database access
- Communication overhead
- Poor resource utilization

6. Improvements

The distributed AI system can be improved through:

- Load balancing
- Caching
- Asynchronous processing
- Database optimization
- Horizontal scaling
- Monitoring and logging

Conclusion

Data-flow and process mapping provide a clear understanding of how information moves through a distributed AI system. It helps identify communication problems, processing bottlenecks, and opportunities for improving scalability and performance.

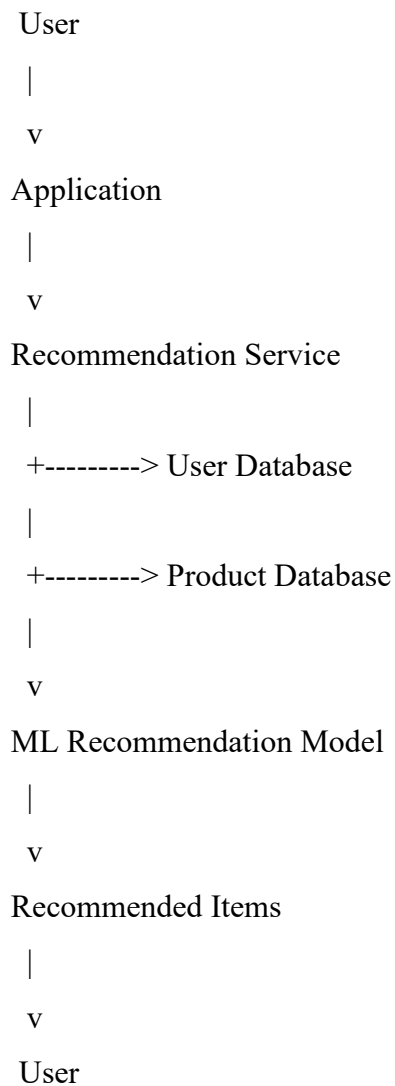
5. Perform reverse engineering on an intelligent system and propose enhancements or redesign strategies to improve scalability, efficiency, and performance.

Reverse engineering is useful for understanding an existing intelligent system and identifying areas where its architecture, algorithms, and processing can be improved. The assessment specifically expects meaningful improvement or redesign suggestions.

Example: Intelligent Recommendation System

A recommendation system suggests products, movies, or other content based on user behavior.

1. Existing Architecture



2. Reverse Engineering Analysis

The system is analyzed to identify:

- Major components
- Data sources
- Recommendation algorithm
- Data flow
- Database operations
- Processing time
- Communication between services

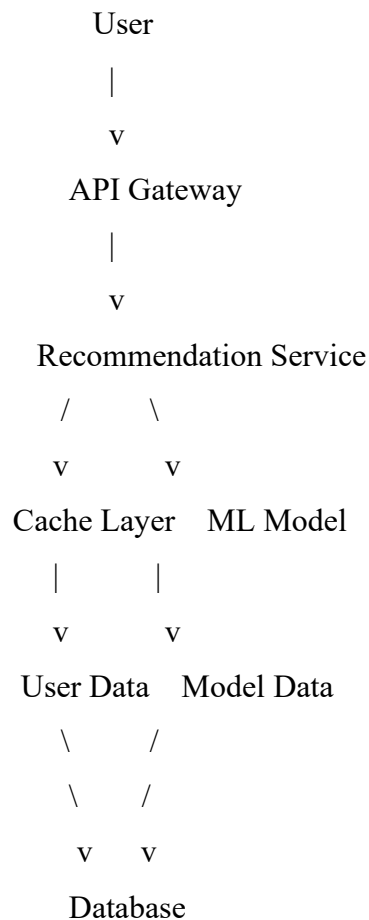
3. Possible Problems

The existing system may have:

- Slow recommendation generation
- Large database queries
- High memory usage
- Poor scalability

- Repeated calculations
- Outdated recommendations

4. Proposed Redesign



5. Scalability Improvements

Load Balancing:

Distribute requests among multiple servers.

Horizontal Scaling:

Add more instances when user traffic increases.

Caching:

Store frequently requested recommendations to reduce repeated computation.

6. Efficiency Improvements

- Optimize database queries.
- Remove unnecessary processing.
- Use efficient data structures.
- Batch similar requests.
- Reduce redundant model inference.

7. AI Model Improvements

The recommendation model can be improved by:

- Using better-quality training data.

- Removing irrelevant features.
- Retraining periodically.
- Comparing multiple algorithms.
- Monitoring prediction accuracy.

8. Performance Monitoring

Important metrics include:

- Response time
- CPU usage
- Memory usage
- Throughput
- Model accuracy
- Error rate

9. Tools and Techniques

Reverse engineering can use:

- System logs
- API monitoring
- Performance monitoring
- Dependency analysis
- Database analysis
- Profiling tools
- Input-output testing

Conclusion

Reverse engineering provides a clear understanding of an intelligent system's existing architecture and limitations. By applying caching, load balancing, optimized databases, efficient algorithms, and scalable architecture, the system can achieve better **scalability, efficiency, reliability, and performance**. This directly addresses the rubric's requirement for meaningful redesign and improvement suggestions.